# Relatório Trabalho RC - Chat P2P

Yuri Shumyatsky - 231012826
Vinicius de Melo Moraes - 231036274
André Ricardo de Faria Santos - 231025889

5 de julho de 2026

## 1 Introdução

Este relatório descreve a implementação de um sistema de chat Peer-to-Peer (P2P). Diferente dos modelos centralizados convencionais, a aplicação adota uma arquitetura descentralizada na qual cada nó atua simultaneamente como cliente e servidor. Para viabilizar a troca de mensagens em tempo real, foi elaborado um protocolo de camada de aplicação próprio, projetado para gerenciar desde o registro de usuários até a manutenção e o encerramento seguro das sessões.

Para contornar o desafio de localização inicial na rede, o projeto utiliza um servidor Rendezvous que funciona exclusivamente como um diretório temporário de nós ativos. Após essa etapa de descoberta, toda a comunicação ocorre de maneira estritamente ponto a ponto. O sistema foi desenvolvido em Python empregando operações assíncronas nativas para garantir o tratamento eficiente de conexões concorrentes.

## 2 Especificação do Protocolo

Foi implementada para o chat uma arquitetura P2P, em que cada usuário comunica-se diretamente com os outros usuários da rede. A existência do servidor Rendezvous serve como um peer de descoberta, no qual os usuários se cadastram e do qual recebem uma lista de peers cadastrados.

Cada peer é identificado por duas informações: name e namespace, que formam um peer_id no formato 'name@namespace', sendo o namespace um agrupador lógico tal qual UnB ou CIC. Mensagens podem ser enviadas de forma única para um peer, para todos os usuários em um namespace, ou como um broadcast para todos os peers cadastrados.

Para conseguir se conectar aos outros peers, é necessário primeiro fazer o registro no servidor Rendezvous ao utilizar uma mensagem do tipo REGISTER. Após registrado, pode ser requisitada uma lista de usuários com um comando de DISCOVER. Por fim, para sair da rede P2P basta utilizar um comando UNREGISTER.

As conexões são iniciadas através de mensagens do tipo HELLO/HELLO_OK, que iniciam a troca de mensagens periódicas de keep-alive do formato PING/PONG. Essas conexões podem ser terminadas ao enviar uma mensagem do tipo BYE e receber BYE_OK.

Uma vez que haja uma conexão estabelecida entre peers, podem ser enviadas mensagens via SEND/ACK ou PUB, que publica uma mensagem para todos os peers especificados ou dentro de um namespace.

Seguem abaixo exemplos de cada um das possíveis mensagens.

```
{
"type": "REGISTER",
"namespace": "UnB",
"name": "grupoG1",
"port": 4000,
"ttl": 3600
}
```

Comando REGISTER

```
{
"type": "DISCOVER",
"namespace": "UnB"
}
```

Comando DISCOVER

```
{
"type": "UNREGISTER",
"namespace": "UnB",
"name": "grupoG1",
"port": 4000
}
```

Comando UNREGISTER

```
{
   "type": "HELLO",
   "peer_id": "grupoG1@UnB",
   "version": "1.0",
   "features": ["ack", "metrics"],
   "ttl": 1
}
```

Comando HELLO

```json
{
    "type": "HELLO_OK",
    "peer_id": "grupoG?@UnB",
    "version": "1.0",
    "features": ["ack", "metrics"],
    "ttl": 1
}
```

Comando HELLO_OK

```json
{
    "type": "PING",
    "msg_id": "uuid",
    "timestamp": "2025-10-27T10:00:00Z",
    "ttl": 1
}
```

Comando PING

```json
{
    "type": "PONG",
    "msg_id": "uuid",
    "timestamp": "2025-10-27T10:00:00Z",
    "ttl": 1
}
```

Comando PONG

```
{
   "type": "SEND",
   "msg_id": "uuid",
   "src": "grupoG1@UnB",
   "dst": "grupoG?@UnB",
   "payload": "Olá!",
   "require_ack": true,
   "ttl": 1
}
```

Comando SEND

```
{
   "type": "ACK",
   "msg_id": "uuid",
   "timestamp": "2025-10-27T10:00:01Z",
   "ttl": 1
}
```

Comando ACK

```
{
   "type": "PUB",
   "msg_id": "uuid",
   "src": "grupoG1@UnB",
   "dst": "#UnB",
   "payload": "Aviso para todos!",
   "require_ack": false,
   "ttl": 1
}
```

Comando PUB

```
{
   "type": "BYE",
   "msg_id": "uuid",
   "src": "grupoG1@UnB",
   "dst": "grupoG?@UnB",
   "reason": "Encerrando sessão",
   "ttl": 1
}
```

Comando BYE

```
{
   "type": "BYE_OK",
   "msg_id": "uuid",
   "src": "grupoG?@UnB",
   "dst": "grupoG1@UnB",
   "ttl": 1
}
```

Comando BYE_OK

## 3 Arquitetura de Implementação

A implementação do projeto foi inteiramente em Python, usando apenas bibliotecas na standard library. Em especial, a biblioteca asyncio permite o uso concorrente de várias conexões em uma única *thread*. Segue abaixo uma visão da estrutura de diretórios do projeto, seguida por uma explicação do funcionamento de cada componente.

**Estrutura de diretórios do projeto:**

/pyp2p-trabalho-RC

```
├── cli.py
├── keep_alive.py
├── main.py
├── message_router.py
├── p2p_client.py
├── peer_connection.py
├── peer_table.py
├── project_config.py
├── project_log.py
├── README.md
├── rendevouz_connection.py
└── state.py
```

**Configurações Dinâmicas**

*project_config.py* é o arquivo responsável por gerenciar as configurações dos protocolos a serem implementados. Ao iniciar, um arquivo json é criado com as configurações iniciais, este arquivo então é utilizado para criar um dicionário do qual os valores de configurações serão obtidos e, caso necessário, modificados. As funções implementadas são "*write_config*", que escreve o json, "*getConfig*", que retorna o dicionário com as chaves e valores das configurações, e "*addConfig*" e "*change_config*", que adicionam ou modificam, respectivamente, o dicionário com as configurações.

**Logs**

*project_log* é a implementação do arquivo de registro durante o uso. Cria um arquivo (*logs/p2p_app.log*) com o histórico de ações tomadas durante todo o código, criando novos logs na pasta caso o tamanho deles ultrapasse o limite de 1MB.

**Message Router**

O message router é implementado através da biblioteca asyncio, uma vez que ela permite o gerenciamento de diversas funções usando uma única thread, assim, permitindo o gerenciamento dos *reader* e *writer*. Sua função principal é enviar mensagens para outros peers e tratar as mensagens que outros peers enviam. As mensagens são enviadas através das funções "*send_message*" para mensagens diretas e "*publish_message*" para um namespace ou mensagens globais, ao mesmo tempo, uma tarefa do asyncio verifica constantemente o

espaço de leitura buscando escritas por outros peers e tratando elas de acordo com o tipo da mensagem recebida.

**Rendezvous**

O *rendevouz_connection* é responsável por gerenciar a interação com o servidor de rendezvous, através das ações "REGISTER", "DISCOVER" e "UNREGISTER", o processo começa com funções de nome igual à ação a ser feita, que, por meio de uma função auxiliar, define um dicionário baseado no protocolo a ser utilizado. A função "REGISTER" inicia a conexão com o servidor, fornecendo valores como IP, porta e usuário (name@namespace), valores que são coletados a partir do arquivo de configuração. "DISCOVER" é utilizado para encontrar peers conectados ao servidor ou dentro de um "namespace" fornecido, retornando um dicionário com os peers correspondentes à requisição. Por fim, o "UNREGISTER" encerra a conexão com o servidor, removendo o usuário da lista de conectados.

**Keep Alive**

"*keep_alive*" é o arquivo responsável por gerenciar a manutenção das conexões TCP entre os peers, a partir de um loop enviando "pings" para todos peers conectados a cada intervalo de tempo definido no arquivo de configuração, e respondendo com "pongs" para pings que são recebidos, caso um número definido de "pings" enviado para um peer não sejam respondidos com "pongs", a conexão com o peer é terminada e ele é marcado como "STALE".

**Peer Table**

O gerenciamento de peers e a tabela contendo os peers conectados ao rendezvous são controlados pelo arquivo *peer_table.py*. Um objeto com a classe "peer_table" é instanciado, contendo um set de peers conectados e outro de peers não conectados, e funções que movem os peers entre sets mediante seu status. Além disso, outra classe, "peer", é instanciada para representar os peers obtidos no "DISCOVER", essa classe possui diversos atributos que são usados para determinar as características dos peers. Além disso, a peer table é mantida em um arquivo gerado chamado de PeerTable.json.

**Peer Connection**

Progredindo da peer table, o arquivo *peer_connection.py* gerencia as conexões com outros peers, tentando conexões com novos peers na tabela e tentando aceitar pedidos de conexão recebidos de outros peers.

**State**

A função "state" é responsável por verificar o estado da tabela de peers, reescrevendo caso ela não tenha sido criada e servindo como um intermediário entre processos na peer table e o loop de registro no rendezvous.

**P2P Client**

Controla os processos paralelos durante a execução e providencia uma saída controlada caso processos estejam em andamento ao receber o comando "/quit"

**Main**

"*main.py*" é o arquivo responsável por inicializar a execução do p2p, ele inicia a criação do arquivo de log, inicia a criação do arquivo de configurações e inicia o loop do "cli" para receber os comandos e se conectar ao rendezvous. É o ponto de entrada do programa.

**Interface de Usuário**

O cliente possui interatividade no formato REPL, implementado no arquivo **cli.py**. Segue uma tabela mostrando as funções aceitas com as quais os usuários podem interagir com o sistema.

| **Comando** | **Função** |
|---|---|
| /peers [* \| #namespace] | Descobrir e listar peers |
| /msg <peer_id><mensagem> | Enviar mensagem direta |
| /pub * <mensagem> | Enviar broadcast global |
| /pub #<namespace><mensagem> | Enviar mensagem para todos do namespace |
| /conn | Mostrar conexões ativas (inbound/outbound) |
| /rtt | Exibir RTT médio por peer |
| /reconnect | Forçar reconciliação de peers |
| /log <Nível> | Ajustar nível de log (DEBUG, INFO...) |
| /quit | Encerrar aplicação |

Tabela 1 - Comandos REPL

## 4 Instruções para Execução

### 4.1 Requisitos Mínimos

- Sistema: Linux (qualquer distribuição com suporte ao Python)
- Python: 3.11 ou superior (testado em 3.12)
- Rede: conectividade TCP para comunicação com o servidor rendezvous e peers
- Permissões: escrita no diretório do projeto (criação de 'config.json' e pasta 'logs')
- Privilégios regulares de usuário para realizar a abertura de sockets

### 4.2 Execução

Como o projeto foi inteiramente implementado em python e usando apenas a standard library, não há necessidade de compilação ou de instalação de bibliotecas de terceiros, portanto basta rodar o código diretamente usando o interpretador da linguagem python.

```
$ git clone https://github.com/Fire1900/pyp2p-trabalho-RC.git
$ cd pyp2p-trabalho-RC
$ python3 main.py
```